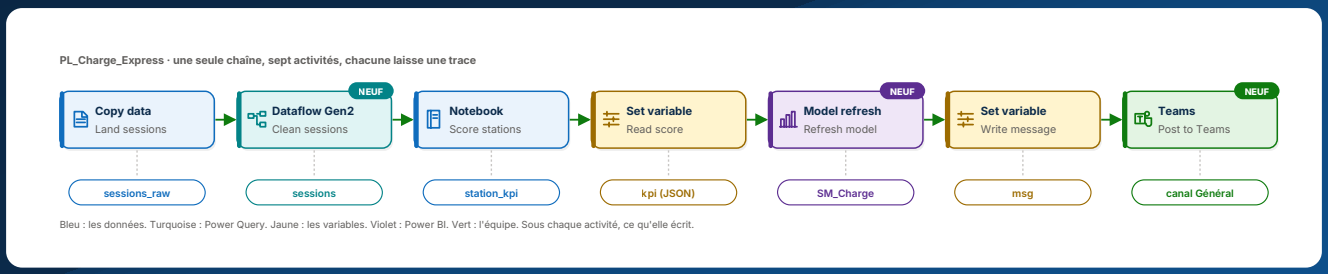
MICROSOFT FABRIC · DATA FACTORY · GUIDE DE RÉFÉRENCE

Pipeline Microsoft Fabric : nettoyer, scorer, publier, prévenir

ATTERRIR NETTOYER SCORER PUBLIER PRÉVENIR

Un pipeline qui fait travailler ensemble les quatre moteurs de Fabric : Data Factory pose le fichier brut, Power Query le nettoie sans une ligne de code, Spark calcule les indicateurs et rend son verdict, Power BI se met à jour, et l'équipe reçoit le bilan dans Teams. Sept activités, une seule chaîne, et un seuil d'alerte que l'on change sans toucher au pipeline.



7	3	4	1 509	1 482
activités	types nouveaux	moteurs Fabric	lignes brutes	sessions propres

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

PRÉREQUIS
Un espace de travail Fabric sur une capacité, Microsoft Teams avec une équipe (ou Outlook à défaut)

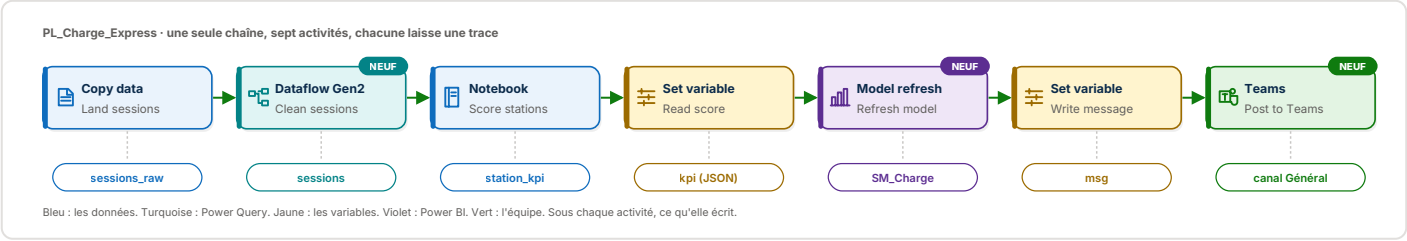
FICHIERS DU PROJET
Le dossier data : le fichier de sessions, le notebook à importer, toutes les expressions

DURÉE
Environ 45 minutes, en quatre étapes vérifiées une à une

VUE D'ENSEMBLE

Ce que vous construisez

Quatre stations de recharge en Moselle, Metz Gare, Metz Technopôle, Thionville et Forbach, exportent chaque semaine leurs sessions de recharge. Le fichier arrive sale : des énergies illisibles, des doublons, des relevés négatifs. Le pipeline le pose tel quel, le nettoie, mesure le taux de sessions en échec de chaque station, met à jour le modèle Power BI et poste dans Teams soit une alerte, soit un « tout va bien ». Trois types d'activité nouveaux : **Flux de données**, **Actualisation du modèle sémantique**, **Teams**.



Les sept activités

ACTIVITÉ	TYPE	RÔLE
`Land sessions`	Copy data	Pose le CSV brut, tel quel, dans la table `sessions_raw` : tout en texte, rien n'est perdu
`Clean sessions`	Flux de données NEUF	Exécute `DF_Clean_Sessions` : types, erreurs, doublons, filtre, durée, vers la table `sessions`
`Score stations`	Notebook	Calcule les indicateurs par station dans `station_kpi` et renvoie un verdict JSON au pipeline
`Read score`	Set variable	Range ce JSON dans la variable `kpi` : lu une fois, réutilisé partout
`Refresh model`	Actualisation du modèle sémantique NEUF	Met à jour `SM_Charge` : le rapport Power BI montre les chiffres de cette exécution
`Write message`	Set variable	Rédige le message : alerte ou « tout va bien », décidé dans l'expression par `if()`
`Post to Teams`	Teams NEUF	Poste le message dans le canal de l'équipe

Le fichier et ses défauts

`sessions_semaine40.csv`, du 28 septembre au 4 octobre 2026, huit colonnes : `session_id`, `station`, `charger_id`, `start_time`, `end_time`, `energy_wh`, `price_cents`, `status` (`completed` ou `failed`).

LIGNE	NOMBRE
Lignes du fichier	**1 509**
`energy_wh` illisible, valeur `ERR`	12
Session envoyée deux fois	9
Énergie négative, capteur déréglé	6
Sessions propres attendues	**1 482**

Pourquoi ce pipeline est différent

CHAQUE MOTEUR À SA PLACE Power Query pour nettoyer, parce que chaque étape se voit et se relit. Spark pour calculer, parce qu'un taux par station se dit en dix lignes. Le pipeline ne fait qu'enchaîner et décider.

UN CONTRAT DE SORTIE Le notebook ne renvoie pas « OK » : il renvoie un JSON avec ses chiffres. Le pipeline le range dans une variable et en tire le message.

REJOUABLE SANS RISQUE Remplacer partout : la table brute, la table propre, les indicateurs. Relancer dix fois donne dix fois 1 482 sessions.

Le pipeline en une image : une cuisine de restaurant

La livraison
Copy data dépose les cartons tels quels en réserve, sans les ouvrir.

Le commis
Power Query trie, lave, jette l'abîmé et les doublons.

Le chef
Le notebook cuisine les chiffres et rend un verdict écrit.

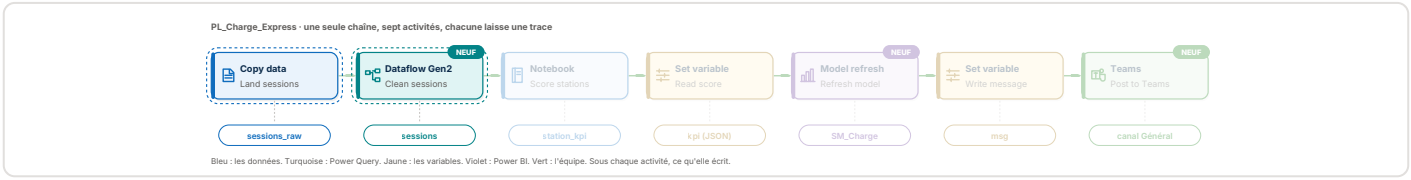
Le menu
Power BI affiche aussitôt les plats du jour.

Le serveur
Teams annonce en salle : alerte, ou tout va bien.

ÉTAPE 1

Poser le fichier, puis le nettoyer avec Power Query

1 · ATTERRIR ET NETTOYER | 2 · SCORER ET MODÉLISER | 3 · ENCHAÎNER | 4 · PRÉVENIR ET EXÉCUTER



1 Le lakehouse et le fichier

1. Espace de travail **Fabric_Pipeline_5**. **Nouvel élément**, **Lakehouse**, nom `LH_Charge`.
2. Dans **Fichiers**, **Nouveau sous-dossier** `inbox`, puis **Charger**, **Charger des fichiers** : `sessions_semaine40.csv`.

2 Land sessions — COPY DATA

1. **Nouvel élément**, **Pipeline**, nom `PL_Charge_Express`. **Activités**, **Copier les données**, **Ajouter au canevas**, nom `Land sessions`.
2. **Source** : `LH_Charge`, racine **Fichiers**, chemin `inbox` / `sessions_semaine40.csv`, format **DelimitedText**, **Première ligne comme en-tête** coché.
3. **Destination** : `LH_Charge`, racine **Tables**, nouvelle table `sessions_raw`, **Action de table** : **Remplacer**. Ne touchez pas au mappage.
4. **Enregistrer**, **Exécuter**. Sortie de l'activité : `rowsCopied` = **1 509**.

> **BRUT VEUT DIRE BRUT** Sans mappage, le Copy écrit toutes les colonnes en texte. Le `ERR` passe sans casser le chargement : c'est au nettoyage de le refuser, pas à l'atterrissage.

3 DF_Clean_Sessions — DATAFLOW GEN2 · NEUF

1. **Nouvel élément**, **Dataflow Gen2**, nom `DF_Clean_Sessions`. **Obtenir des données**, **Lakehouse**. Connexion : **Compte professionnel**, **Se connecter**, **Connexion**. Puis `LH_Charge`, table `sessions_raw`, **Créer**.
2. Dans l'éditeur Power Query, six gestes, dans cet ordre :
   1. `energy_wh` et `price_cents` (Ctrl+clic), clic droit, **Modifier le type**, **Nombre entier**.
   2. `start_time` et `end_time`, **Modifier le type**, **Date/heure**.
   3. Ctrl+A sur les en-têtes, **Accueil**, **Supprimer les lignes**, **Supprimer les erreurs** : les 12 `ERR` partent.
   4. `session_id` seul, **Supprimer les lignes**, **Supprimer les doublons** : 9 lignes partent.
   5. Flèche de `energy_wh`, **Filtres de nombre**, **Supérieur ou égal à** `0` : 6 lignes partent.
   6. **Ajouter une colonne**, **Colonne personnalisée**, nom `duration_min`, type **Nombre décimal**, formule :

```
Duration.TotalMinutes([end_time] - [start_time])
```

3. En bas à droite, **Destination des données**, **+**, **Lakehouse**, `LH_Charge`, **Nouvelle table** `sessions`, méthode **Remplacer**.
4. **Enregistrer et exécuter**. Quelques minutes plus tard, `sessions` apparaît dans le lakehouse : **1 482** lignes, neuf colonnes.

AVANT LE NOTEBOOK ☐ `sessions_raw` : 1 509 lignes ☐ `sessions` : 1 482 lignes ☐ `duration_min` est décimale ☐ le flux est enregistré

ÉTAPE 2

Scorer les stations avec Spark, ouvrir un modèle Power BI

1 · ATTERRIR ET NETTOYER | 2 · SCORER ET MODÉLISER | 3 · ENCHAÎNER | 4 · PRÉVENIR ET EXÉCUTER

4 NB_Score_Stations

NOTEBOOK · IMPORTÉ, RIEN À TAPER

1. Dans l'espace de travail, **Importer**, **Notebook**, **À partir de cet ordinateur** : `NB_Score_Stations.ipynb`. Ouvrez-le.
2. Panneau **Explorateur**, **Ajouter des sources de données**, **Lakehouse existant**, `LH_Charge` : il devient le lakehouse par défaut.
3. Vérifiez que la première cellule porte l'étiquette **Cellule de paramètres**. Sinon : **…** de la cellule, **Basculer la cellule de paramètre**.
4. **Exécuter tout**. La dernière ligne affiche le verdict, et la table `station_kpi` apparaît : quatre lignes.

```
threshold = 0.10
run_id = "manuel"

import json
from pyspark.sql import functions as F

s = spark.read.table("sessions")
k = (s.groupBy("station")
      .agg(F.count("*").alias("sessions"),
           F.sum(F.when(F.col("status") == "failed",
1).otherwise(0)).alias("failed"),
           F.round(F.sum("energy_wh") / 1000, 1).alias("energy_kwh"),
           F.round(F.sum("price_cents") / 100,
2).alias("revenue_eur"),
           F.round(F.avg("duration_min"), 1).alias("avg_minutes"))
      .withColumn("failure_rate", F.round(F.col("failed") /
F.col("sessions"), 3))
      .withColumn("alert", F.col("failure_rate") >
F.lit(float(threshold)))
      .withColumn("run_id", F.lit(run_id))
      .withColumn("scored_at", F.current_timestamp()))
k.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("station_kpi")

rows = k.collect()
worst = max(rows, key=lambda r: r["failure_rate"])
result = {"sessions": sum(r["sessions"] for r in rows),
          "energy_kwh": round(sum(r["energy_kwh"] for r in rows)),
          "alerts": sum(1 for r in rows if r["alert"]),
          "worst": worst["station"],
          "worst_pct": round(worst["failure_rate"] * 100, 1)}
print(result)
notebookutils.notebook.exit(json.dumps(result))
```

LA CELLULE DE PARAMÈTRES Lancé à la main, le notebook prend `threshold = 0.10`. Lancé par le pipeline, ces deux valeurs sont remplacées par celles que le pipeline envoie : c'est ainsi qu'on change le seuil sans ouvrir le code.

LE VERDICT RENVOYÉ

```
{"sessions": 1482, "energy_kwh": 38396,
"alerts": 1, "worst": "THIONVILLE",
"worst_pct": 12.7}
```

POURQUOI UN JSON
`notebookutils.notebook.exit()` ne renvoie qu'une chaîne. En y mettant un JSON, le notebook renvoie cinq valeurs d'un coup, que le pipeline relit avec `json()`.

5 SM_Charge

MODÈLE SÉMANTIQUE DIRECT LAKE

1. Ouvrez `LH_Charge`. Ruban, **Nouveau modèle sémantique**, nom `SM_Charge`.
2. Cochez `sessions` et `station_kpi`, **Confirmer**. Le modèle s'ouvre : rien d'autre à faire.

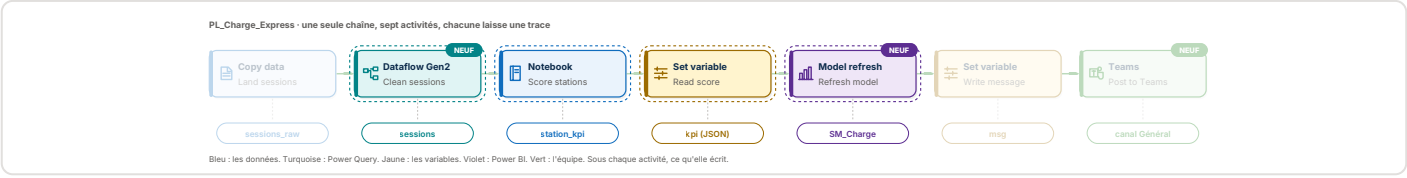
DIRECT LAKE Le modèle lit les fichiers Delta du lakehouse sans les copier. L'actualiser ne recharge rien : il pointe simplement vers la dernière version des tables. Quelques secondes, quel que soit le volume.

AVANT D'ENCHAÎNER ☐ `station_kpi` : 4 lignes ☐ Thionville à 0.127 ☐ `SM_Charge` existe ☐ le notebook a son lakehouse

ÉTAPE 3

Enchaîner le flux, le notebook et le modèle dans le pipeline

1 · ATTERRIR ET NETTOYER | 2 · SCORER ET MODÉLISER | **3 · ENCHAÎNER** | 4 · PRÉVENIR ET EXÉCUTER



6 Paramètre et variables

1. Rouvrez `PL_Charge_Express`, cliquez sur le fond du canevas. Onglet **Paramètres**, **Nouveau** :

Nom	`alert_threshold`
Type	`Float`
Valeur par défaut	`0.1`

2. Onglet **Variables**, deux variables de type `String` : `kpi` et `msg`.

7 Clean sessions

FLUX DE DONNÉES · NEUF

1. **Activités**, **Flux de données**, nom `Clean sessions`, relié à la sortie verte de `Land sessions`.
2. **Paramètres** : espace de travail courant, flux `DF_Clean_Sessions`.

8 Score stations

NOTEBOOK

1. **Notebook**, nom `Score stations`, relié derrière `Clean sessions`. **Paramètres** : **Connexion** laissée vide (elle est facultative), espace `Fabric_Pipeline_5`, bloc-notes `NB_Score_Stations`.
2. **Paramètres de base**, **Nouveau**, deux fois :

threshold · Float	`@pipeline().parameters.alert_threshold`
run_id · String	`@pipeline().RunId`

Les noms doivent être ceux de la cellule de paramètres, à la lettre près.

9 Read score

SET VARIABLE

1. **Définir une variable**, nom `Read score`, relié derrière `Score stations`. **Paramètres**, nom `kpi`, valeur :

```
@activity('Score stations').output.result.exitValue
```

LIRE UNE FOIS Sans cette variable, chaque expression suivante répéterait le chemin complet vers la sortie du notebook. Avec elle, on écrit `json(variables('kpi')).alerts`, et si le notebook change de nom, une seule ligne bouge.

10 Refresh model

MODÈLE SÉMANTIQUE · NEUF

1. **Activités**, **Actualisation du modèle sémantique**, nom `Refresh model`, relié derrière `Read score`.
2. **Paramètres**, **Connexion** : **Nouveau**, type **Compte d'organisation**, **Se connecter**. Espace `Fabric_Pipeline_5`, modèle `SM_Charge`.
3. **Table(s)** : laissez vide, tout le modèle est actualisé. Dans **Avancé**, cochez **Attendre la fin** s'il apparaît ; sinon l'activité attend déjà par défaut.
4. **Enregistrer**.

LE RAPPORT SUIT LE PIPELINE Placé après le notebook, l'actualisation garantit qu'un rapport ouvert sur `SM_Charge` montre les chiffres de cette exécution, jamais ceux de la veille.

AVANT LE MESSAGE ☐ cinq activités reliées en vert ☐ `threshold` en Float ☐ `kpi` lit `exitValue` ☐ Enregistrer, le point a disparu

ÉTAPE 4

Rédiger le message, le poster dans Teams, exécuter

1 · ATTERRIR ET NETTOYER | 2 · SCORER ET MODÉLISER | 3 · ENCHAÎNER | **4 · PRÉVENIR ET EXÉCUTER**

11 Write message — SET VARIABLE

1. **Définir une variable**, nom `Write message`, derrière `Refresh model`. Variable `msg`, valeur : le bloc de la ligne **[Write message]** de `expressions.txt`, reproduit sous les deux colonnes.

LA DÉCISION DANS L'EXPRESSION Pas de Condition Si : `if()` choisit entre deux phrases. Une seule activité Teams suffit ensuite, et le pipeline reste une ligne droite.

12 Post to Teams — TEAMS · NEUF

1. **Activités**, **Microsoft Teams**, nom `Post to Teams`, derrière `Write message`. **Connexion** : connectez votre compte.
2. **Publier dans** : **Canal**, votre équipe, canal **Général**. **Message** : l'éclair, puis `variables('msg')`. **Enregistrer**.

```
@if(greater(json(variables('kpi')).alerts, 0), concat('Alerte bornes : ', string(json(variables('kpi')).alerts), ' station(s) au-dessus du seuil de ', string(pipeline().parameters.alert_threshold), '. La pire : ', json(variables('kpi')).worst, ' avec ', string(json(variables('kpi')).worst_pct), ' % de sessions en échec.'), concat('Bornes de recharge : aucune station au-dessus du seuil de ', string(pipeline().parameters.alert_threshold), '. ', string(json(variables('kpi')).sessions), ' sessions, ', string(json(variables('kpi')).energy_kwh), ' kWh livrés.'))
```

OBJECTIF
Deux exécutions de `PL_Charge_Express`, même fichier, deux seuils : une alerte, puis un « tout va bien ».

RÉSULTAT
1 509 lignes brutes, 1 482 sessions, 4 stations scorées, 2 messages dans Teams.

Exécution 1 — alert_threshold = 0.1

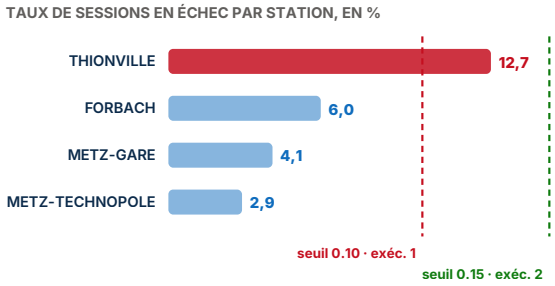
Les sept activités passent au vert. Thionville dépasse le seuil. Dans Teams :

```
Alerte bornes : 1 station(s) au-dessus du seuil de 0.1. La pire : THIONVILLE avec 12.7 % de sessions en échec.
```

Exécution 2 — alert_threshold = 0.15

Mêmes 1 509 et 1 482 lignes : tout est remplacé, rien n'est doublé. Dans Teams :

```
Bornes de recharge : aucune station au-dessus du seuil de 0.15. 1482 sessions, 38396 kWh livrés.
```



OÙ LIRE CHAQUE PREUVE Dans **Sortie** de l'exécution : `rowsCopied` sur `Land sessions`, `exitValue` sur `Score stations`, la valeur de `msg` sur `Write message`. Puis le message dans le canal Teams.

Trois réflexes d'expert sur ce pipeline

Planifier la semaine

Planifier dans le ruban du pipeline : chaque lundi à 6 h. Le seuil reste à 0.1 par défaut ; pour un audit plus souple, une seconde planification passe 0.15.

Prévenir aussi l'échec

Une seconde activité Teams reliée par le fil **rouge** de `Score stations`. Une seule flèche rouge : plusieurs flèches vers une même activité exigent qu'elles échouent toutes.

Tracer l'exécution

La colonne `run_id` de `station_kpi` porte l'identifiant de l'exécution du pipeline : on retrouve dans le moniteur quelle exécution a produit quel chiffre.

RÉFÉRENCE

Vérifier, retrouver une expression, débloquer

Vérifier dans le lakehouse

Ouvrez `LH_Charge`, basculez sur le **Point de terminaison SQL**, **Nouvelle requête SQL** :

```
SELECT station, sessions, failed,
  failure_rate, energy_kwh,
  revenue_eur, avg_minutes, alert
FROM station_kpi
ORDER BY failure_rate DESC;
```

STATION	SESS.	ÉCHECS	TAUX	KWH	€
THIONVILLE	379	48	**0.127**	9 519,0	3 712,45
FORBACH	215	13	0.06	5 583,5	2 177,62
METZ-GARE	509	21	0.041	12 570,3	4 902,33
METZ-TECHNOPOLE	379	11	0.029	10 723,6	4 182,13

1 482
sessions propres sur 1 509 lignes reçues

12,7 %
d'échecs à Thionville, trois fois Metz Gare

38 396
kWh livrés sur la semaine

14 974,53 €
facturés par les quatre stations

Les expressions

OÙ	EXPRESSION
DF_Clean_Sessions, colonne `duration_min`	`Duration.TotalMinutes([end_time] - [start_time])`
Score stations, `threshold`	`@pipeline().parameters.alert_threshold`
Score stations, `run_id`	`@pipeline().RunId`
Read score, `kpi`	`@activity('Score stations').output.result.exitValue`
Write message, `msg`	Le bloc de la page 6, ou la ligne [Write message] de `expressions.txt`
Post to Teams, Message	`@{variables('msg')}`, inséré par l'éclair

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Le flux ne voit pas `sessions_raw`	Le Copy n'a pas encore tourné. Exécutez d'abord le pipeline avec la seule activité `Land sessions`.
Toute la colonne `start_time` part en erreur	Clic droit, **Modifier le type**, **Utilisation des paramètres régionaux** : **Date/heure**, **Anglais (États-Unis)**.
`sessions` n'a pas 1 482 lignes	L'ordre des gestes compte : types, puis erreurs, puis doublons sur `session_id` seul, puis filtre. Reprenez dans le volet **Étapes appliquées**.
Notebook : « Table or view not found: sessions »	Pas de lakehouse par défaut. **Explorateur**, ajoutez `LH_Charge`, relancez.
Le seuil du pipeline est ignoré	La cellule de paramètres n'est pas marquée, ou le paramètre de base est mal nommé. `threshold` et `run_id`, à la lettre.
« Informations d'identification non valide » dans le flux	L'ancienne connexion Lakehouse a expiré. Liste **Connexion**, **Créer une connexion**, **Compte professionnel**, **Se connecter**.
« did not have accessToken specified »	Le type **Identité de l'espace de travail** exige une identité que l'espace n'a pas. Choisissez **Compte d'organisation**, ou laissez la connexion du Notebook vide.
`Write message` : « json » ou « property alerts » introuvable	`kpi` est vide : le notebook n'a pas atteint `exit()`. Lisez la sortie de `Score stations`.
Expression coupée après un copier-coller	Le PDF ajoute des espaces aux retours à la ligne. Copiez depuis `expressions.txt`, une ligne entière.
Pas d'équipe, ou Teams indisponible	Dans Teams, **Créer une équipe** `Data Ops`. Sans Teams : remplacez par **E-mail Office 365**, **Corps** = `variables('msg')`.